

Burning a Firmware Image

The flint utility enables you to burn the Flash from a binary image. To burn the entire Flash from a raw binary image, use the following command line:

```
1 # flint -d <device> -i <fw-file> [-guid <GUID> | -guids <4 GUIDS> | -mac <MA
```

where:



device	Device on which the flash is burned.
fw-file	Binary firmware file.
GUID(s)	Optional, for InfiniBand adapters and 4th generation (Group I) switches. One or four GUIDs. • If 4 GUIDS are provided (-guids flag), they will be assigned as node, Port 1, Port 2 and sys- tem image GUIDs, respectively. • If only one GUID is provided (-guid flag), it will be assigned as node GUID. Its values +1, +2 and +3 will be assigned as Port 1, Port 2 and system image GUID, respectively. • If no -guid/-guids flag is provided, the current GUIDs will be preserved on the device. NOTE: For 4th generation (Group I), four GUIDs must be specified but Ports 1 and 2 GUIDs are ignored and should be set to 0. NOTE: A GUID is a 16-digit hexadecimal number. If less than 16 digits are provided, leading zeros will be inserted.
MAC(s)	Optional, for Ethernet and InfiniBand adapters and switches. • If 2 MACs are provided (-macs flag), they will be assigned to Port 1 and Port 2, respectively. • If only one MAC is provided (-mac flag), it will be assigned to Port 1; MAC+1 will be assigned to Port 2. • If no -mac/-macs flag is provided, the current LIDs will be preserved on the device. NOTE: A MAC is a 12-digit hexadecimal number. If less than 12 digits are provided, leading zeros will be inserted.

To burn a firmware image:

1. Update the firmware on the device, keeping the current GUIDs and VSD. (Note: This is the common way to use the tool.)

```
1 # flint -d /dev/mst/mt4099_pci_cr0 -i fw-4099-2_42_5000-MCX354A-FCB_A2.
```

2. Update the firmware on the device, specifying the GUIDs to burn.

```
1 # flint -d /dev/mst/mt4099_pci_cr0 -i fw-4099-2_42_5000-MCX354A-FCB_A2.
```

3. Update the firmware on the device, specifying the MACs to burn.

```
1 # flint -d /dev/mst/mt4099_pci_cr0 -i fw-4099-2_42_5000-MCX354A-FCB_A2.
```

4. Burn the image on a blank Flash device. This means that no GUIDs are currently burnt on the device, therefore they must be supplied (with -guid/-guids) by the burning command. Moreover, the burn process cannot be failsafe when burning a blank Flash, therefore the -nofs flag must be specified.

```
1 burn# flint -d /dev/mst/mt4099_pci_cr0 -i fw-4099-2_42_5000-MCX354A-FCB_A2.
```

5. Read FW from the device and save it as an image file.

```
1 # flint -d /dev/mst/mt4099_pci_cr0 ri Flash_Image_Copy.bin
```

6. MT58100 SwitchX switch:

Burn the image on a blank Flash device. Meaning, no GUIDs/MACs are currently burnt on the device, therefore they must be supplied (with -guid/-guids and -mac/-macs) by the burning command. Moreover, the burn process cannot be failsafe when burning a blank Flash, therefore the -nofs flag must be specified.

```
1 # flint -d /dev/mst/mtusb-1 -i /tmp/fw-sx.bin -nofs -guids 000002c90000
```

7. MT58100 SwitchX switch inband firmware update:

```
1 # flint -d lid-0x18 -i /tmp/fw-sx.bin b
```

Burning the MFA2 Images

Burning the MFA2 images enables the user to extract (i.e. unzip) 4MB images from MFA2 archive that matches the device type and device PSID. If there are more than one matching images, the user may use the `--latest_fw` flag and burn the latest firmware, or choose the required image from the user menu.

The device flash **MUST** have all relevant device information (signatures, PSID, VPD, DEV_INFO, MFG_INFO, etc.) valid since MFA2 format does not have that information and without the burn process will fail.

```
flint -d <device> -i <mfa2 file> --psid <PSID string> (optionally) --latest_fw (optionally) -silent (optionally) b (or burn)
```

- **Burning the MFA2 Images when the Device Includes a Valid Image**

In this scenario, the user **may** (optional) provide a “`-psid`” flag and extract from the MFA2 archive the image that matches this flag, and this way actually change the PSID on the device.

- **Burning the MFA2 Images when in Live Fish Mode**

In this scenario, the user **must** provide a “`-psid`” flag and extract from the MFA2 archive the image that matches this flag, and this way actually change the PSID on the device.

Cable Firmware Update (In-Field-Firmware-Update)

This capability is supported only in NVIDIA Quantum switch systems and hosts with NVIDIA ConnectX-6 adapter cards.

The In-Field-Firmware-Update (IFFU) tool works via the switches/NICs in the datacenters and is intended for remote control. The tool is used to update cables transceivers' firmware.

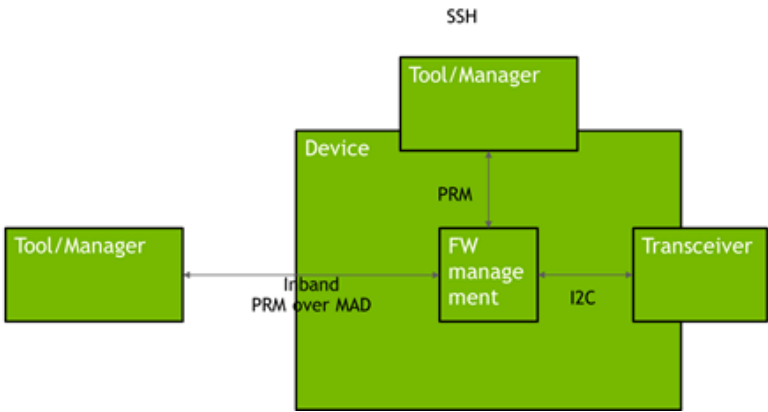
Optical Cables and Transceivers are active network components which run firmware, and as any component running firmware, the ability to update firmware is mandatory. Transceiver firmware update is a system flow which requires the following elements:

- Tool/Manager which will perform the firmware update
- Switch/NIC firmware management used as a middleman between the Manager and the cable transceiver
- Transceiver firmware: target for upgrade

The figure below shows the tool/manager which runs on a remotely controlled host or (in case of managed switches) on a switch, shown as 'Device'.

The manager can query the transceivers type and the current running firmware to understand if an update is required. When an update is required, the manager can apply set of commands that will send the remote host device a new firmware images for the specific transceiver(s) and activate a firmware update flow. The set of commands is defined with low

level primitives to support full flexibility for the user. High level script can be applied on top of the manager and allow system wide update.



The update of modules/AOCs connected to switches is done over InfiniBand (inband) PRM registries. Whereas, the update of modules connected to NICs is done over MCC (RegAccess) on the host.

Inband connection implies that unmanaged switches like QM8790 support IFFU. Each device (NIC, Switch) can update only the modules connected directly to it, not the far end. Updating the far end transceiver/end of the AOC requires the same operation to be done at the far end switch(es).

The Tool/Manager host must have MST rev. 4.16.00 or later installed. Remote control from outside the cluster (data center) requires access to the host being used as Tool/Manager. When the cluster has many switches, multiple hosts may be engaged in the upgrade process. The host(s) can be remotely controlled via VNC access.

Firmware Burning Across a Cluster (Data Center)

The IFFU function described below works on one switch. Cluster-wide firmware updating is done by use of a script which initiates the update procedure in multiple switches in parallel by initiating an instance of the flint command for each switch. In large clusters the script can be executed on multiple hosts, each handling a different part of the cluster.

Cable Burn Command

```
1 # flint -d <device> --linkx <flags> <commands>
```

where:

Flags:



<device>	The name of the target switch (one only).
--downstream_device_id_start_index <downstream_device_id_start_index>	The port number of the first LinkX cable/transceiver. (min. port number = 1)

--num_of_downstream_devices <num_of_downstream_devices>	the number of cables/transceivers to burn. They are burnt sequentially.
--linkx_auto_update	Use this flag to burn all supported cables/transceivers connected to the switch.
--download_transfer	Use this flag to perform download and transfer of all cable data for cables. Download and transfer are not performed by default. This flag is only relevant for cable components.
--activate	Use this flag to apply the activation of the new firmware in the updated devices. Activation is not performed by default.
--activate_delay_sec <timeout in seconds>	Use this flag to activate all cable devices connected to host with delay, acceptable values are between 0 and 255 (default - 1, immediately). Important: 'activate' flag must be set. This flag is relevant only for cable components.
--i <image>	'i' indicates 'binary Image' followed by the path and file name of the bin file to download into the cable/transceiver.
--downstream_device_ids <list of ports>	Use this flag to specify the LNKX ports to perform query. List must be only comma- separated numbers, without spaces

Commands:



b[urn]	Burn flash
q[uey]	Query misc. flash/firmware characteristics.

Updating the Firmware

Burning a firmware cable transceiver connected to the host (NIC or switch) is done using the "flint" tool. To do so, the user should use the "--linkx" flag.

Firmware can be burnt in follow one of the methods:

- **Burn with Auto-update:**

1. Transfer the data from the host.

```
1 # flint -d <device> --linkx --linkx_auto_update --download_transfer -i
```

Example:

```
1 # flint -d lid-2 --linkx --linkx_auto_update --download_transfer -i ima
```

2. Activate the firmware.

```
1 # flint -d <device> --linkx --linkx_auto_update --activate b
```

The flint "--activate" flag behavior is changed to include a minimal delay of 1 second to avoid disconnections if the connected port is being activated. To use the "legacy" activation flow, use the "--activate_delay_sec 0" command.

Example:

```
1 # flint -d lid-2 --linkx --linkx_auto_update --activate b
```

Activate with delay Example:

```
1 # flint -d lid-2 --linkx --linkx_auto_update --activate --activate_delay
```

Transfer and Activate Example:

```
1 # flint -d lid-2 --linkx --linkx_auto_update --download_transfer --acti
```

Burning all cables in an unmanaged switch in one operation is risky. If the cables do not link up after the update, you lose connection to the switch – permanently.

Burn half of the cables, check that they come up after burning, then burn the other half.

- **Burning multiple cables in the switch using the 'Range':**

1. Transfer the data from the host.

```
1 # flint -d <device> --linkx --downstream_device_id_start_index <index>
```

2. Activate the firmware.

```
1 # flint -d <device> --linkx --downstream_device_id_start_index <index>
```

Example of Download Transfer with Activation, range indices is 10 to 16:

```
1 # flint -d lid-2 --linkx --downstream_device_id_start_index 10 --num_of
```

This will update 6 AOCs/Transceivers starting from port 10, i.e. all ports in the range 10...15.

You cannot 'overburn' the same firmware version into a transceiver/AOC as the one already installed. This is to prevent wasting time re-burning transceivers in a large cluster. If you try to burn the existing FW version, the command responds:

Cable burn failed, error is LinkX downstream transfer failed for device index i

Example of successful update of 1 AOC:

```
1 -I- Downloading FW ...
2 FSMST_INITIALIZE - OK
3 Writing COMPID_LINKX component - OK
4 FSMST_LOCKED - OK
5 FSMST_DOWNSTREAM_DEVICE_TRANSFER - OK
6 FSMST_LOCKED - OK
7 Please wait while activating the transceiver(s) FW ...
8 FSMST_ACTIVATE - OK..]
9 -I- Cable burn finished successfully.
```

Downloading and burning takes approx. 1½ minute + activation ½ minute for one cable. The time for multiple cables depends on which ports they are plugged into.

Querying Firmware Version from an Image

Querying a cable image for firmware version is done using the "flint" tool.

```
1 # flint -i <fw file> q
```

Querying Vendor Specific Firmware Information from a NVIDIA AOC / Transceiver

Querying a firmware cable transceiver is done using the "flint" tool.

In case the Vendor Specific query command is not support by the firmware, it will run the CMIS standard query implemented by the firmware.

```
1 # flint -d <cable device> q
```

Querying Firmware Information from an AOC / Transceiver

Querying a firmware cable transceiver connected to the host (NIC or switch) is done using the "flint" tool. To do so, the user should use the "--linkx" flag.

```
1 # flint -d <device> --linkx --downstream_device_ids <ids> [--output_file <fi
```

Query ports 1,2,5 Example:

```
1 # flint -d <device> --linkx --downstream_device_ids 1,2,5 q
```

The system responds with information about the firmware version loaded into the transceivers.

The firmware version of all cables plugged into ports 1...40 of a switch with lid #nn can alternatively be checked with the mlxlink command:

```
1 # for i in {1..40}; do echo $i; mlxlink -d lid-nn -p $i -m | grep 'Part\\|FW'
```

Checking successful burning and operation - Example:

It is essential to check that the links come up AFTER the cable FW is updated and reactivated. This can be done as follows:

```
1 # for i in {36..40}; do echo $i; mlxlink -d lid-nn -p $i -m | grep 'Part\\|FW'
```

The 'State' parameter was added to the query. The response has the following format (example):

```
# 36
```

3	State	: Active
4		
5	Vendor Part Number	: MFS1S00-H010
6		
7	FW Version	: 38.100.59
8		
9	37	
10		
11	State	: Active
12		
13	Vendor Part Number	: MFS1S00-H010
14		
15	FW Version	: 38.100.59
16		
17	38	
18		
19	State	: Active
20		
21	Vendor Part Number	: MFS1S00-H010
22		
23	FW Version	: 38.100.59
24		
25	39	
26		
27	State	: Active
28		
29	Vendor Part Number	: MFS1S00-H010
30		
31	FW Version	: 38.100.59
32		
33	40	
34		
35	State	: Active
36		
37	Vendor Part Number	: MFS1S00-H010
38		
39	FW Version	: 38.100.59

Last updated: November 07, 2022



[Privacy Policy](#)

[Your Privacy Choices](#)

[Terms of Service](#)

[Accessibility](#)

[Corporate Policies](#)

[Product Security](#)

[Contact](#)

Copyright © 2026 · Powered by Scroll Sites & Atlassian Confluence